

Sprint 4

Nivel 1

Ejercicio 1

El Country Manager de Alemania necesita revisar urgentemente las transacciones del día 12 de marzo de 2022.

Tarea:

- » 1. Escribe la consulta que une (JOIN) transacciones i compañías.
- » 2. Filtra los resultados per la fecha indicada y el país "Germany".
- » 3. Sin ejecutar la consulta, realiza un "Dry Run" (auditoria de costes).

 **Observación:** Fíjate que BigQuery lee casi toda la tabla aunque solo se pide un día (Full Table Scan).

```
1 SELECT c.company_name,t.* FROM
2 `sprint3-analytics-borja-b.sprint3_silver.companies_clean` c
3 INNER JOIN `sprint3-analytics-borja-b.sprint3_silver.transactions_clean` t
4 ON c.company_id=t.business_id
5 WHERE c.country = 'Germany'
6 AND t.timestamp = '2022-03-12';
```

 Esta consulta procesará 11.16 MB cuando se ejecute.

Para este ejercicio, selecciono el nombre de la compañía de la tabla companies_clean, y selecciono todo de la tabla transactions. Uno ambas tablas con un inner join a través de sus claves, y finalmente hago una restricción para que filtre por un país en concreto (Alemania) y por una fecha en concreto (12 de marzo de 2022).

Como se ve a pie de captura de imagen, "Dry run" nos indica que la consulta procesará 11.16MB. Esto se debe a que no existe ninguna partición creada en la tabla de la columna timestamp, lo cual implica que, aunque se solicite una fecha en concreto, el sistema debe leer la tabla entera para encontrar los datos que coincidan con la restricción (Full Table Scan).

En caso de existir una partición en el campo timestamp, el consumo de la consulta disminuiría drásticamente ya que sabría donde buscar los datos requeridos, sin necesidad de leer toda la tabla.

Ejercicio 2

Paso 1: Generación de Datos Recientes (Mocking Data)

Crea una tabla intermedia llamada `sprint3_silver.transactions_recent` a partir de la tabla `sprint3_silver.transactions_clean`. Tu objetivo es mantener todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que entre en los últimos 50 días.

» Pista Técnica: para hacerlo sin escribir todas las columnas a mano, puedes usar `SELECT *EXCEPT(timestamp)`. Para calcular la fecha aleatoria, deberás de restar una cantidad de días al instante actual.

Te serán muy útiles las funciones `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, y la combinación `CAST(RAND() * 50 AS INT64)` para generar los días de desfase.

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-borja-b.sprint3_silver.transactions_recent` AS
2 SELECT
3   * EXCEPT(timestamp),
4   TIMESTAMP_SUB(
5     CURRENT_TIMESTAMP(),
6     INTERVAL CAST(RAND() * 50 AS INT64) DAY
7   ) AS timestamp
8 FROM `sprint3-analytics-borja-b.sprint3_silver.transactions_clean`;
```

✓ Esta consulta procesará 10.39 MB cuando se ejecute.

Creo la nueva tabla `transaction_recent` y hago un `SELECT * EXCEPT(timestamp)`. Esto lo que hace es copiar la tabla `transactions_clean` (citada más abajo con el `FROM`) a excepción del campo `timestamp`.

Por separado, le digo que substraiga un intervalo concreto de fechas con `TIMESTAMP_SUB()`, como parámetros le paso la fecha actual con `CURRENT_TIMESTAMP` y le resto el intervalo solicitado con un `CAST` y el método `RAND` multiplicado por 50 días, lo paso a entero con el `cast` y lo renombro a `timestamp`. Finalmente, también le pongo un `DAY`

al Interval, para que el sistema sepa que, el número creado aleatoriamente, va a ser días. En caso de haber puesto HOUR, el sistema entendería horas.

Ejecuto la query y reviso que se haya creado correctamente:

The screenshot shows the Google BigQuery console. On the left, the 'Dataset Explorer' pane lists various tables, with 'transactions_recent' under the 'sprint3_silver' dataset highlighted with a red rectangle. The main editor shows a SQL query that creates or replaces a table named 'transactions_recent' in the 'sprint3-analytics-borja-b.sprint3_silver' dataset. The query uses a SELECT statement with a subquery that generates a random timestamp within a 50-day interval. Below the query editor, a status bar indicates 'Se completó la consulta' (Query completed) and 'Se está usando la cuota de procesamiento según demanda' (Using on-demand processing quota). The 'Results' tab is active, showing a message: 'Esta declaración creó una nueva tabla con el nombre transactions_recent.' (This statement created a new table with the name transactions_recent).

Paso 2: Creación de la Tabla Optimizada (Partitioning & Clustering)

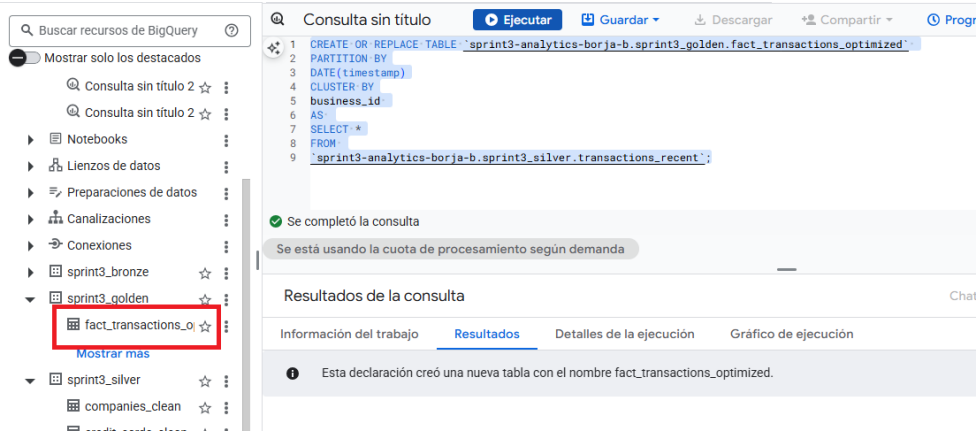
Ahora, crea la tabla definitiva `sprint3_gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent`. Has de construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenaje:

- » **Particionamiento (Partitioning):** Divide la tabla por la fecha del campo `DATE(timestamp)`. Esto permitirá que las consultas que filtren por día (`WHERE date = ...`) solo lean la partición necesaria.
- » **Clustering:** Ordena los datos dentro de cada partición por `business_id`. Esto acelerará drásticamente los filtros por compañía y los cruzará (`JOIN`) con la dimensión de empresa.

The screenshot shows the Google BigQuery console with a SQL query editor. The query is a DDL statement to create or replace a table named 'fact_transactions_optimized' in the 'sprint3-analytics-borja-b.sprint3_golden' dataset. The query specifies partitioning by 'DATE(timestamp)' and clustering by 'business_id'. The data is selected from the 'sprint3-analytics-borja-b.sprint3_silver.transactions_recent' table. Below the query editor, a status bar indicates 'Esta consulta procesará 11.16 MB cuando se ejecute.' (This query will process 11.16 MB when executed) and 'Se está usando la cuota de procesamiento según demanda' (Using on-demand processing quota).

Creo la tabla `fact_transactions_optimized` en la capa Golden. Hago la partición por DATE y la columna timestamp, y agrupo (Cluster) por `business_id`. Finalmente, le digo que copie la tabla `transactions_recent`.

El “Dry run” muestra un consumo de 11.16MB, igual que en la consulta anterior. Esto se debe a que estoy creando una tabla nueva con los datos de la antigua, por tanto el sistema debe leer y escanear toda la tabla, es por ello que su consumo corresponde al total de la tabla. BigQuery tiene un almacenamiento columnar, y al hacer select all, tiene que leerlas todas. Sin embargo, al hacer las particiones y agrupar por business_id, hará que las consultas en dicha tabla sean mucho más ligeras.



Compruebo que se haya creado correctamente, y busco en los detalles de la tabla, que la partición se ha creado:

Esquema		Detalles	Vista previa	Explorador de t
Etiquetas				
Claves primarias				
Etiquetas				
Tipo de tabla		Particionada		
Particionada por		DAY		
Particionada en el campo		timestamp		
Vencimiento de la partición		60 días		
Filtro de partición		No es obligatorio		
Agrupada por		business_id		

Esquema

Detalles

Vista previa

Explorador de tablas

Vista previa

Estadísticas

Linaje

Perfil de datos

Calidad de los datos

Fila	transaction_id	card_id	business_id	amount	declined	product_ids	user_id
1	AB445F17-6425-4711-9E81-050...	CcS-9223	b-2222	447.73	0	15	4642
						98	
						99	
						7	
2	236AF17C-171A-4DC7-A233-AB...	CcS-5274	b-2222	143.04	0	82	693
						58	
3	148B750F-3786-44F8-9E13-EA5...	CcS-5535	b-2222	2.12	0	66	954
4	B89390ED-AB26-40B1-B7D4-D0...	CcS-4919	b-2222	136.96	0	76	338
						4	
5	EE4DA605-12B8-4AFA-88CB-97...	CcS-4983	b-2222	141.66	0	11	402
						20	
6	AF5484A0-B272-497A-89EB-27...	CcS-6612	b-2222	488.05	0	1	2031
						22	

Resultados por página: 501 - 50 de 100000|<<>>

Ejercicio 3

Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costes que has conseguido. Simularemos una petición típica de negocio: “queremos ver todas las transacciones de los últimos 30 días”.

El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas diferentes y comparar el coste computacional.

Construye una consulta SQL que seleccione todas las columnas (SELECT*) y filtre los datos de los últimos 30 días.

No ejecutes las consultas aún. En este ejercicio solo usaremos el validador de BigQuery (“Dry run”) para visualizar los Bytes procesador de manera gratuita.

Haz la comparativa siguiendo estos dos pasos:

Paso 1 (tabla no optimizada): apunta tu consulta a la tabla `sprint3_silver.transactions_recent`. Observa el validador y haz una captura de pantalla de los bytes procesados.

```
1 SELECT * FROM `sprint3-analytics-borja-b.sprint3_silver.transactions_recent`
2 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

✓ Esta consulta procesará 11.16 MB cuando se ejecute.

Esta es la consulta a la tabla no optimizada transactions_recent en la capa silver. Vemos que el consumo son 11.16MB. Al usar BigQuery un motor columnar, debe leer todas las columnas de la tabla solicitada, incluyendo la tabla timestamp. Al no haber particiones, el sistema debe revisar toda la tabla.

Paso 2 (tabla optimizada): sin cambiar nada de la lógica de tu consulta, modifica solo el FROM para que ahora apunte a la tabla particionada sprint3_gold.fact_transactions_optimized. Observa como cambia el validador y haz la segunda captura.

```
1 SELECT * FROM `sprint3-analytics-borja-b.sprint3_golden.fact_transactions_optimized`
2 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
```

✓ Esta consulta procesará 6.84 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Realizo la misma consulta, pero esta vez a la tabla optimizada (particionada). El consumo se reduce hasta 6.84MB, ya que el sistema tan solo lee los datos que contiene la condición anidada en el WHERE, en lugar de leer la tabla entera.

Esto supone una reducción de 4,32MB entre las dos tablas, lo que supone un ahorro del 38,7% en recursos.

Ejercicio 4

Crea una Vista Materializada llamada `sprint3_gold.mv_daily_sales` que muestre las ventas totales por día.

```
1 CREATE MATERIALIZED VIEW `sprint3-analytics-borja-b.sprint3_golden.mv_daily_sales` AS
2 SELECT
3   SUM(amount) AS total_vendes,
4   DATE(timestamp) AS fecha
5 FROM `sprint3-analytics-borja-b.sprint3_golden.fact_transactions_optimized`
6 GROUP BY fecha;
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Creo la vista materializada como indica el enunciado, y selecciono la suma de amount, que nombro como total_vendes, y extraigo la fecha de la columna timestamp, que anteriormente habíamos particionado, selecciono la tabla de la que quiero coger los datos (fact_transactions_optimized) y agrupo por fecha.

En la select de la columna timestamp, le añado la función DATE() para que coja solo la fecha dejando de lado la hora de la venta, ya que de otra manera lo contabilizaría por separado una misma venta en el mismo día.

Sale que la consulta procesará 0B. Esto es por que la query lo que hace es crear una vista, pero no usará recursos hasta que no se invoque y se hagan consultas sobre ella. Lo que hace la vista materializada es guardar el código y el resultado, pero a la vez, se actualiza automáticamente, por lo que los datos nuevos introducidos se suman al histórico.

Reviso que se haya creado correctamente:

The screenshot shows the Google Cloud BigQuery interface. On the left, the 'Revisar' (Review) sidebar lists various resources, with 'mv_daily_sales' highlighted in a red box. The main area displays the SQL query used to create the view, which matches the one in the exercise. Below the query, a green checkmark indicates 'Se completó la consulta' (Query completed). The 'Resultados de la consulta' (Query results) section shows a message: 'Esta declaración creó una nueva vista materializada denominada mv_daily_sales.' (This statement created a new materialized view named mv_daily_sales.)

Filtro Escribir el nombre o valor de la propiedad						
☐	Nombre del campo	Tipo	Modo	Descripción	Clave	Interacción
☐	total_vendes	FLOAT	NULLABLE	-	-	-
☐	fecha	DATE	NULLABLE	-	-	-

```

1 SELECT
2 ROUND(total_vendes,2) AS total_vendes,
3 fecha
4 FROM `sprint3-analytics-borja-b.sprint3_golden.mv_daily_sales`
5 ORDER BY fecha DESC;

```

✓ Esta consulta procesará 816 B cuando se ejecute.

Para mostrar los datos de la vista, hago la query en la que selecciono todo de la vista, y ordeno por fecha descendiente. La consulta consume 816B. Es un valor extremadamente bajo, que ocupa menos de 1KB, demostrando la eficiencia del Smart Catching. Se traduce en una optimización y ahorro de recursos de hasta un 99% respecto a la tabla original no optimizada. Ejecuto la sentencia y este es el resultado:

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila		total_vendes	fecha			
1		270670.85	2026-06-10			
2		512799.73	2026-06-09			
3		534761.08	2026-06-08			
4		520969.67	2026-06-07			
5		505396.88	2026-06-06			
6		522884.04	2026-06-05			
7		508979.59	2026-06-04			
8		531500.18	2026-06-03			
9		503441.53	2026-06-02			
10		516835.99	2026-06-01			
11		507841.57	2026-05-31			
12		512816.85	2026-05-30			
13		534438.78	2026-05-29			
14		520094.8	2026-05-28			

Resultados por página: 50 1 - 50 de 51

Nivel 2

Ejercicio 1

El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos “VIP” como aquellos con un gasto acumulado superior a 500€. Necesitamos un informe que, por cada VIP, muestre su nombre, contacto y su patrón de compra: cuantas veces ha comprado, cuanto gasta de media y cual fue su compra récord.

Tarea:

- » 1. Crea una CTE que se llame **VIP_Stats** que agrupe por usuario i calcule:
 - El Gasto Total (SUM).
 - La Cantidad de Transacciones (COUNT).
 - El Tiquet Medio (AVG), redondeando a 2 decimales.
 - La Compra Máxima (MAX).
 - Filtro: Mantén solo aquellos el Gasto Total de los que sea > 500.
- **Cruza la CTE con users_combined para obtener los datos personales.**

Requisitos de Salida:

- Columnas: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenado per total_gastat descendente.

```

1 WITH VIP_Status AS(
2   SELECT
3     ROUND(SUM(t.amount),2) AS total_gastat,
4     COUNT(t.transaction_id) AS num_compres,
5     ROUND(AVG(t.amount),2) AS tiquet_mitja,
6     MAX(t.amount) AS compra_maxima,
7     u.user_id,
8     u.name,
9     u.surname,
10    u.email
11  FROM `sprint3-analytics-borja-b.sprint3_silver.transactions_recent` AS t
12  INNER JOIN `sprint3-analytics-borja-b.sprint3_silver.users_combined` AS u
13    ON t.user_id = u.user_id
14  WHERE t.declined = '0'
15  GROUP BY
16    u.user_id,
17    u.name,
18    u.surname,
19    u.email
20  HAVING total_gastat > 500
21 )
22 SELECT
23   user_id,
24   CONCAT(name, ' ', surname) AS nom_complet,
25   email,
26   total_gastat,
27   num_compres,
28   tiquet_mitja,
29   compra_maxima
30 FROM VIP_Status
31 ORDER BY total_gastat DESC;

```

✓ Esta consulta procesará 5.47 MB cuando se ejecute.

Creo el CTE con la query WITH VIP_Status AS y selecciono la suma del amount como total_gastat, cuento los transaction_id como num_compres, el average de amount redondeado a dos decimales como tiquet_mitja, el máximo amount como compra_máxima de la tabla transactions_recent, y luego los campos user_id, name, surname, email de la tabla users_combined.

Uno ambas tablas con un INNER JOIN y sus claves, hago una restricción en la que busque solo las transacciones que han sido aprobadas y agrupo por los campos user_id, name, surname, email para luego restringir a las compras mayores a 500€ con un having.

Fuera del CTE, hago un select de la tabla creada VIP_Status de los campos user_id, concateno nombre y apellido para que aparezcan bajo el mismo campo como nom_complet, email total_gastat, num_compres, tiquet_mitja, compra_maxima y ordeno por el total_gastado descendente.

Veo que la consulta procesará 5,47MB. Al utilizar una CTE BigQuery genera esta tabla temporal directamente en la memoria RAM. Tan solo existe durante la ejecución de la query, y se destruye automáticamente al mostrar los resultados. De esta manera se optimizan los recursos de memoria y costes.

Resultado:

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
Fila	user_id	nom_complet	email	total_gastat	num_compres	tiquet_mitja	compra_maxima	
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	40669.16	91	446.91	554.71	
2	318	Bnyr Astuw	bnyr.astuw@example.com	38614.39	86	449.0	615.51	
3	455	Fwfll Phylgnmhnd	fwfll.phylgnmhnd@example.com	34491.27	66	522.59	724.44	
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	30796.3	73	421.87	675.78	
5	185	Molly Gilliam	donec@outlook.couk	29394.79	107	274.72	659.31	
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	29249.51	70	417.85	630.52	
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	25485.05	67	380.37	646.0	
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	23711.9	78	304.0	604.9	
9	349	Yxcqjk Fnceqbikxmt	yxcqjk.fnceqbikxmt@example.c...	23504.47	53	443.48	675.05	
10	317	Qikcvc Nvficifqdh	qikcvc.nvficifqdh@example.com	18991.45	54	351.69	600.77	
11	388	Rbjupnac Cfjyidx	rbjupnac.cfjyidx@example.com	18432.84	62	297.3	479.03	
12	319	Fxtg Eedgl	fxtg.eedgl@example.com	17770.39	62	286.62	576.22	
13	443	Qimzjbfj Cvpdvtz	qimzjbfj.cvpdvtz@example.com	17310.42	75	230.81	524.33	
14	393	Nxbq Bmbzxiihu	nxbq.bmbzxiihu@example.com	16726.47	76	220.09	534.22	
15	437	Kxnl Rzhpnaz	kxnl.rzhpnaz@example.com	16260.46	50	325.21	572.37	
16	183	Germane Whitehead	malesuada.malesuada.integer...	14818.17	76	194.98	624.64	

Resultados por página: 50 1 – 50 de 5000

Ejercicio 2

La Dirección Financiera quiere monitorizar la “velocidad de ventas” diaria. Necesitamos un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).

Para este cálculo NO has de usar la tabla de hechos original ni recalcular sumas. Como buen Data Engineer, debes de usar la Vista Materializada `sprint3_gold.mv_daily_sales` que creaste en el ejercicio anterior. Esto garantiza que el análisis de tendencias sea instantáneo y no consuma recursos innecesarios.

Tarea:

Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

- » **1.Data:** La fecha de venta.
- » **2.Vendes_Avui:** El total vendido aquel día.
- » **3.Vendes_Ahir:** El total vendido el día anterior (usa funciones ventana).
- » **4.Diff_Percentual:** El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeando a 2 decimales.



Pista Técnica:

Utiliza la función `LAG(campo) OVER (ORDER BY data)` para leer el valor de la fila anterior sin necesidad de hacer JOINS complejos.

```
1 WITH Sales_History AS(
2   SELECT
3     fecha AS data,
4     total_vendes AS Vendes_Avui,
5     LAG(total_vendes) OVER (ORDER BY fecha ASC) AS Vendes_Ahir
6   FROM `sprint3-analytics-borja-b.sprint3_golden.mv_daily_sales`
7 )
8 SELECT
9   data AS Data,
10  ROUND(Vendes_Avui,2) AS Vendes_Avui,
11  ROUND(Vendes_Ahir,2) AS Vendes_Ahir,
12  ROUND(SAFE_DIVIDE(Vendes_Avui - Vendes_Ahir, Vendes_Ahir) * 100, 2) AS Diff_Percentual
13 FROM Sales_History
14 ORDER BY Data ASC;
```

✓ Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Creo una tabla CTE llamada Sales_History y selecciono de la vista materializada mv_daily_sales la fecha que renombro a data, el total_vendes que nombro Vendes_Avui, y uso una función ventana, junto a la función LAG() que coge la columna total_vendes, y las ordena por fecha de manera ascendente. La renombro Vendes_Ahir. La función LAG() permite acceder a datos de una fila anterior a la consultada. Por último, selecciono la vista materializada de donde elegir los datos.

Una vez creada la CTE, hago un select con los campos data, Vendes_Avui, Vendes_Ahir y aquí es donde hago la función que generará la diferencia porcentual de ventas por día. Con la función SAFE_DIVIDE, cojo el campo Vendes_Avui y le resto Vendes_Ahir, este resultado lo divido por las Vendas_Ahir y multiplico por 100. Finalmente, lo envuelvo todo con un ROUND() para redondear a 2 decimales y renombro a Diff_Percentual. Elijo la CTE Sales_History y ordeno por Data Asc (de más antiguo a más reciente).

Cómo veremos en la captura de pantalla, el SAFE_DIVIDE lo que hace es, que en el primer registro, como no tendrá un número anterior al que comparar y no se puede dividir por 0, que en su lugar ponga un null, ya que de otra manera, la consulta arrojaría un error.

Resultado:

Fila	Data	t0_	f1_	Diff_Percentual
1	2026-04-21	236681.37	null	null
2	2026-04-22	501893.17	236681.37	112.05
3	2026-04-23	521278.1	501893.17	3.86
4	2026-04-24	529075.26	521278.1	1.5
5	2026-04-25	522997.64	529075.26	-1.15
6	2026-04-26	507232.63	522997.64	-3.01
7	2026-04-27	521469.14	507232.63	2.81
8	2026-04-28	497596.06	521469.14	-4.58
9	2026-04-29	523984.42	497596.06	5.3
10	2026-04-30	524417.73	523984.42	0.08
11	2026-05-01	495059.19	524417.73	-5.6
12	2026-05-02	502074.93	495059.19	1.42
13	2026-05-03	531237.68	502074.93	5.81
14	2026-05-04	522077.0	531237.68	-1.72
15	2026-05-05	533217.16	522077.0	2.13
16	2026-05-06	512966.97	533217.16	-3.8
17	2026-05-07	505853.51	512966.97	-1.39

Resultados por página: 50 1 - 50 de 51

Ejercicio 3

El Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuanto hemos facturado en total desde el inicio del año hasta hoy (YTD – Year To Date).

Tarea:

Usando la vista materializada `mv_daily_sales` (para no recalcular agregaciones base), genera un informe con tres columnas:

- » 1.Data.
- » 2.Vendes del Dia: Redondeando a 2 decimales.
- » 3.Vendes Acumulades YTD: Una suma progresiva de les ventas que ha de reiniciarse cada 1 de Enero. El resultado final ha redondearse a 2 decimales.

Pista Técnica:

Para conseguir el reinicio anual, necesitas dividir la ventana usando `PARTITION BY EXTRACT (YEAR FROM data)`. La estructura completa sería: `SUM(...) OVER(PARTITION BY---ORDER BY---ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

```
1 WITH History_Sales AS(
2   SELECT
3     fecha AS Data,
4     ROUND(total_vendes,2) AS Vendes_del_Dia
5   FROM `sprint3-analytics-borja-b.sprint3_golden.mv_daily_sales`
6 )
7 SELECT
8   Data,
9   Vendes_del_Dia,
10  ROUND(
11    SUM(Vendes_del_Dia) OVER(
12      PARTITION BY EXTRACT (year FROM Data) ORDER BY Data ASC
13      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
14    ),2
15  ) AS Vendes_Acumulades_YTD
16 FROM History_Sales
17 ORDER BY Data ASC;
18
```

✓ Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Creo una CTE con el nombre `History_Sales` y selecciono de la vista materializada `mv_daily_sales` las columnas `fecha`, que renombro a `Date`, y hago un redondeo a 2 decimales del campo `total_vendes`, que renombro a `Vendes_del_Dia`.

Fuera del CTE, hago otro select de la tabla creada donde selecciono la `Data`, `Vendes_del_Dia`, y aquí hago la función ventana donde sumo las cantidades de `Vendes_del_Dia` y hago una partición por año del campo `Data`, y lo ordeno por el mismo campo `ASC` (de más antiguo a más reciente). Con **“ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW”** lo que hago es decir que sume desde el inicio del año, hasta la fecha más reciente. Finalmente, envuelvo la función con un `ROUND()` que

redondea a 2 decimales, lo renombro como Vendes_Acumulades_YTD y lo ordeno de manera ascendente.

Resultado:

Resultados de la consulta Chat Guardar los resultados ▾ Abrir en ▾ ↕

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	Data ▾	Vendes_del_Dia ▾	Vendes_Acumula...			
1	2026-04-21	236681.37	236681.37			
2	2026-04-22	501893.17	738574.54			
3	2026-04-23	521278.1	1259852.64			
4	2026-04-24	529075.26	1788927.9			
5	2026-04-25	522997.64	2311925.54			
6	2026-04-26	507232.63	2819158.17			
7	2026-04-27	521469.14	3340627.31			
8	2026-04-28	497596.06	3838223.37			
9	2026-04-29	523984.42	4362207.79			
10	2026-04-30	524417.73	4886625.52			
11	2026-05-01	495059.19	5381684.71			
12	2026-05-02	502074.93	5883759.64			
13	2026-05-03	531237.68	6414997.32			
14	2026-05-04	522077.0	6937074.32			

Resultados por página: 50 ▾ 1 – 50 de 51 |< < > >|

Ejercicio 4

El equipo de Márketing lanza la campaña “A la tercera va la vencida”. Queremos enviar un regalo a los usuarios que hayan completado su tercera transacción. No obstante, el presupuesto del regalo no es fijo: depende de la “calidad” del cliente en sus inicios. Necesitamos saber el Tiquet Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo premium.

Tarea:

Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

- » 1.Dades d'usuari (user_id, nom_complet, email).
- » 2. El dato del importe exacto de la 3ª compra.
- » 3.Medias de las 3 primeras: La media de gasto de sus transacciones 1, 2 i 3.

Pista Técnica:

- » Has de usar la función ROW_NUMBER() para establecer el orden cronológico de las compras.
- »Has de usar la clausula QUALIFY para filtrar.
- » Reto de Optimización: Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial es costoso. Intenta filtrar las filas necesarias antes de realizar la agregación final.

```

1 WITH Client_VIP AS(
2   SELECT
3     u.user_id,
4     CONCAT(u.name, ' ', u.surname) AS nom_complet,
5     u.email,
6     t.amount AS import_3a_compra,
7     AVG(t.amount) OVER(PARTITION BY u.user_id ORDER BY t.timestamp ASC) AS compra_media,
8     t.timestamp AS data_3a_compra,
9     ROW_NUMBER() OVER(PARTITION BY u.user_id ORDER BY t.timestamp ASC) AS num_compra
10    FROM `sprint3-analytics-borja-b.sprint3_silver.users_combined` AS u
11   INNER JOIN `sprint3-analytics-borja-b.sprint3_silver.transactions_recent` AS t
12    ON u.user_id = t.user_id
13   WHERE t.declined = '0'
14 )
15 )
16 SELECT
17   user_id,
18   nom_complet,
19   email,
20   ROUND(compra_media,2) AS compra_media,
21   import_3a_compra,
22   data_3a_compra,
23 FROM Client_VIP
24 QUALIFY ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY data_3a_compra ASC) = 3

```

✓ Esta consulta procesará 2.61 MB cuando se ejecute.

Creo una CTE que se llama Client_VIP que recoja datos de las tablas users_combined y transactions_recent. Seleccione el user_id, concateno el nombre y apellido para que aparezcan en conjunto en una columna y renombro a nom_complet, email, el amount y lo renombro a import_3a_compra para usarlo más adelante, hasta ahora contiene todos los importes; hago un average del amount y una función ventana con una partición por user_id y ordenado por timestamp ascendente, renombro a compra_media, timestamp renombrado a data_3a_compra, y uso la función ROW_NUMBER para numerar las filas de la partición que renombro como num_compra. Uno ambas tablas con sus claves y añado una restricción con where para que coja solo las transacciones aprobadas.

Fuera del CTE, hago un select de los campos user_id, nom_complet, email, compra_media, import_3a_compra, data_3a_compra de la tabla CTE Client_VIP, y uso la función qualify para filtrar el resultado de la función analítica ROW_NUMBER() igualándolo a 3.

Ejecuto y muestro el resultado:

Fila	user_id	nom_complet	email	compra_media	import_3a_compra	data_3a_compra
1	1	Zeus Gamble	interdum.enim@protonmail.edu	361.7	277.83	2026-05-03 09:40:42.154775 UTC
2	10	Robert Mccarthy	fermentum@protonmail.com	152.61	74.64	2026-04-25 09:40:42.154775 UTC
3	100	Melodie Mclean	rius.varius@google.ca	177.28	273.83	2026-05-01 09:40:42.154775 UTC
4	1000	Amkivr Qbulnbp	amkivr.qbulnbp@example.com	218.48	124.04	2026-04-26 09:40:42.154775 UTC
5	1001	Nfvrb Oydaivbg	nfvrb.oydaivbg@example.com	294.96	190.5	2026-05-02 09:40:42.154775 UTC
6	1002	lbfmd Jbddzhvp	lbfmd.jbddzhvp@example.com	198.77	147.53	2026-05-03 09:40:42.154775 UTC
7	1003	Uycilg Sfbdmzj	uycilg.sfbdmzj@example.com	126.28	356.04	2026-05-06 09:40:42.154775 UTC
8	1004	Yjqurq Ojzvgqi	yjqurq.ojzvgqi@example.com	216.67	110.03	2026-04-24 09:40:42.154775 UTC
9	1005	Mnlqtu Glofegwk	mnlqtu.glofegwk@example.com	342.49	59.8	2026-04-24 09:40:42.154775 UTC
10	1006	Rbkocq Swkmynuj	rbkocq.swkmynuj@example.com	220.24	219.66	2026-05-06 09:40:42.154775 UTC
11	1007	Ehcrfp Xahkzrlm	ehcrfp.xahkzrlm@example.com	184.21	422.69	2026-04-25 09:40:42.154775 UTC
12	1008	Securp Faofvqly	securp.faofvqly@example.com	372.29	234.22	2026-04-24 09:40:42.154775 UTC
13	1009	Rsznah Vloffaqt	rsznah.vloffaqt@example.com	212.78	157.9	2026-04-29 09:40:42.154775 UTC
14	101	Sarah Beck	vitaerisus@aol.co.uk	248.64	130.59	2026-04-25 09:40:42.154775 UTC
15	1010	Dvugzj Aaesjmca	dvugzj.aaesjmca@example.com	194.27	249.18	2026-04-28 09:40:42.154775 UTC
16	1011	Vosctr Cpmnpofu	vosctr.cpmnpofu@example.com	130.83	175.79	2026-04-25 09:40:42.154775 UTC
17	1012	Quodit Quodit	quodit.quodit@example.com	319.18	218.89	2026-05-02 09:40:42.154775 UTC

Nivel 3

Ejercicio 1

Actualmente, en la tabla de transacciones, los productos vendidos están “escondidos” en un Array (una lista de IDs). El equipo de ventas quiere analizar qué productos específicos se venden más, pero no pueden hacerlo si los datos están comprimidos en una lista. Necesitamos una Tabla Detallada (Flat Table) donde cada fila represente un producto vendido, aunque signifique que el ID de la transacción se repita.

Tarea:

Crea la tabla `dim_transactions_flat` desnormalizando la información. Has de explotar el array de productos y cruzarlo con el catálogo maestro para obtener los nombre y precios individuales.

⚙️ Pasos Técnicos:

- » 1. Utiliza `CROSS JOIN UNNEST(product_ids)` para transformar el array en filas individuales.
- » 2. Realiza un `JOIN` con la tabla `products` para obtener el nombre (`name`) y precio (`Price`) de cada ítem.

🔗 *Nota: Ve con cuidado con el tipo de datos al hacer el JOIN (Array de INT64 vs Product_ID STRING).*

- » 3. No agrupar el resultado. Queremos ver el desglose línea por línea.

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-borja-b.sprint3_golden.dim_transactions_flat` AS(  
2 SELECT  
3   t.transaction_id,  
4   t.timestamp,  
5   t.amount AS total_ticket,  
6   product_id_aplanado AS product_sku,  
7   p.name AS product_name,  
8   p.price AS product_price,  
9   p.category  
10  FROM `sprint3-analytics-borja-b.sprint3_silver.transactions_recent` AS t  
11  CROSS JOIN UNNEST(t.product_ids) AS product_id_aplanado  
12  INNER JOIN `sprint3-analytics-borja-b.sprint3_silver.products_clean` AS p  
13  ON product_id_aplanado = p.product_id  
14 )
```

✓ Esta consulta procesará 7.09 MB cuando se ejecute.

Creo la tabla `dim_transactions_flat` y selecciono los campos de las tablas que uniré más tarde (las tablas a unir son `transactions_recent` y `products_clean`). Recojo los campos `transaction_id`, `timestamp`, `amount` (que renombro a `total_ticket`), `product_id_aplanado` que renombro a `producto_sku` (este campo es el `product_ids`

anidado en un array, que ya ha sido explotado con un UNNEST y renombrado). De la tabla products_clean cojo los campos name que renombro a producto_name, Price que renombro a producto_price y category.

Selecciono las tablas, y antes de hacer el JOIN, uso el CROSS JOIN UNNEST del campo producto_ids, que es el que agrupa el conjunto de ids en un array. Renombro a product_id_aplanado, y así los ids aparecen desglosados en filas en lugar de texto. Por último, uno ambas tablas con sus claves con un ON, y en caso de la tabla transactions_recent, uso el product_id_aplanado como clave, ya que el sistema ya no entiende producto_ids como llave, al haber sido renombrada.

Ejecuto y reviso que se haya creado correctamente:

Buscar recursos de BigQuery

Mostrar solo los destacados

Notebooks

Lienzos de datos

Preparaciones de datos

Canalizaciones

Conexiones

sprint3_bronze

sprint3_golden

dim_transactions_fl

fact_transactions_r

mv_daily_sales

Consulta sin titulo

Ejecutar

Guardar

Programa

Compartir

Archivo

Editar

Ejecutar

Herramientas

1 CREATE OR REPLACE TABLE `sprint3-analytics-borja-b.sprint3\_golden.dim\_transactions\_flat` AS(
2 SELECT
3 t.transaction_id,
4 t.timestamp,
5 t.amount AS total_ticket,
6 product_id_aplanado AS product_sku,
7 p.name AS product_name,
8 p.price AS product_price,
9 p.category
10 FROM `sprint3-analytics-borja-b.sprint3\_silver.transactions\_recent` AS t
11 CROSS JOIN UNNEST(product_ids) AS product_id_aplanado

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Filtro

Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción
☐	transaction_id	STRING	NULLABLE	-
☐	timestamp	TIMESTAMP	NULLABLE	-
☐	total_ticket	FLOAT	NULLABLE	-
☐	product_sku	INTEGER	NULLABLE	-
☐	product_name	STRING	NULLABLE	-
☐	product_price	FLOAT	NULLABLE	-
☐	category	STRING	NULLABLE	-

Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje	Perfil de datos	Calidad de los datos
Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price	category	
1	5DD05F4-B475-40C4-B395-C3...	2026-04-21 09:40:42.154775 UTC	2.12	66	mustafar jinn	2.12	Books	
2	3B2064FA-B969-4415-BC57-C5...	2026-04-21 09:40:42.154775 UTC	67.19	66	mustafar jinn	2.12	Books	
3	0B03013C-4891-4FCA-AC96-14...	2026-04-21 09:40:42.154775 UTC	111.79	66	mustafar jinn	2.12	Books	
4	3319318D-1FC3-48ED-915E-7A...	2026-04-21 09:40:42.154775 UTC	169.19	66	mustafar jinn	2.12	Books	
5	4151D689-547D-4FD7-81C0-28...	2026-04-21 09:40:42.154775 UTC	109.29	66	mustafar jinn	2.12	Books	
6	6E1A14CE-4252-42DA-B811-22...	2026-04-21 09:40:42.154775 UTC	74.3	66	mustafar jinn	2.12	Books	
7	608455AC-9014-4006-8BA8-6A...	2026-04-21 09:40:42.154775 UTC	187.07	66	mustafar jinn	2.12	Books	
8	8164E0DD-0A13-4C94-A1C4-04...	2026-04-21 09:40:42.154775 UTC	238.8	66	mustafar jinn	2.12	Books	
9	F286A663-06BB-4491-9F66-FF5...	2026-04-21 09:40:42.154775 UTC	330.58	66	mustafar jinn	2.12	Books	
10	11D1CF43-DF74-4515-9A7F-94...	2026-04-21 09:40:42.154775 UTC	321.52	66	mustafar jinn	2.12	Books	
11	D2E89B71-15A9-41CA-8DA0-C9...	2026-04-21 09:40:42.154775 UTC	123.39	66	mustafar jinn	2.12	Books	
12	581247AA-864A-45E5-AFA2-53...	2026-04-21 09:40:42.154775 UTC	170.61	66	mustafar jinn	2.12	Books	
13	6CB3F928-1D07-401F-9864-75...	2026-04-21 09:40:42.154775 UTC	296.41	66	mustafar jinn	2.12	Books	
14	1BE3E3A7-5DE1-4C26-8396-09...	2026-04-21 09:40:42.154775 UTC	219.6	66	mustafar jinn	2.12	Books	
15	011BEEB1-1C9F-49EA-AB17-1C...	2026-04-21 09:40:42.154775 UTC	156.62	66	mustafar jinn	2.12	Books	
16	6625ACA7-3CD0-4F20-A7D5-7D...	2026-04-21 09:40:42.154775 UTC	113.17	66	mustafar jinn	2.12	Books	

Resultados por página: 50 1 – 50 de 253391

Ejercicio 2

Ahora que has preparado la infraestructura y dispones de la tabla `dim_transactions_flat` (donde cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desanidamiento para responder preguntas básicas de negocio.

Tarea:

Genera el Top 5 de productos más vendidos en la historia de la compañía.

 Método:

Consulta directamente tu nueva tabla desnormalizada (`dim_transactions_flat`).

- » Al ser la tabla plana, simplemente necesitas una agrupación estándar (GROUP BY) para el nombre del producto y un recuento (COUNT) de filas.

Archivo
Editar
Ejecutar
Herramientas

```

1 SELECT product_name,COUNT(product_sku) AS total_vendido
2 FROM `sprint3-analytics-borja-b.sprint3_golden.dim_transactions_flat`
3 GROUP BY product_name
4 ORDER BY total_vendido DESC
5 LIMIT 5;

```

 Esta consulta procesará 5.99 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Para conocer el top 5 de productos más vendidos, selecciono el `product_name`, y hago un conteo de los `product_sku` (id) y renombro a `total_vendido`. Selecciono la tabla creada en el anterior ejercicio `dim_transactions_flat`, agrupo por `product_name`, ordeno por el total vendido descendente (que muestre de mayor a menor), y lo limito a 5 para cumplir con el enunciado.

Esta consulta sirve para conocer cómo se desarrollan las ventas por productos, cuáles funcionan mejor y en qué número total de ventas se mueve el negocio.

Ejecuto y este es el resultado:

Resultados de la consulta			
Información del trabajo		Resultados	Visualización
Fila	product_name	total_vendido	
1	Winterfell	10071	
2	the duel	10059	
3	dooku solo	10022	
4	Karstark Dorne	7694	
5	skywalker ewok	7607	

Ejercicio 3

El informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requerimientos urgentes:

- » 1. **Lógica Centralizada:** quieren ver el precio con IVA (21%) desglosado. No obstante, como que el impuesto puede cambiar por país o año, te prohíben poner en el cálculo $\text{precio} * 1,21$ directamente en el código (“Hardcoded”). Has de usar una función reutilizable.
- » 2. **Frescor de Datos:** el tablón ha de actualizarse automáticamente cada mañana a las 7:00 AM antes de la reunión diaria de ventas.

 Tarea:

Has de evolucionar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su generación.

 Pasos Técnicos:

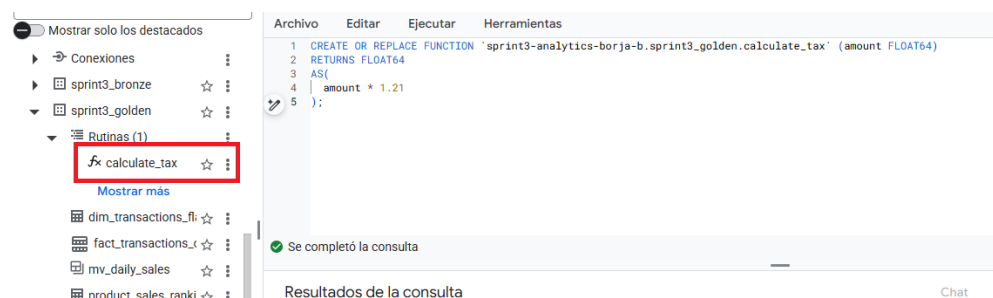
- » 1. **User Defined Functions (UDF):** crea una función SQL persistente llamada `calculate_tax(amount)` que reciba un valor numérico y retorne el resultado aplicando el 21% del impuesto.
- » 2. **Integración y Orquestración:**
 - Modifica el código de creación de la tabla (del ejercicio 1) para que use la nueva función `calculate_tax` y genere una columna nueva: `producto_price_tax_inc`.

- **Configura una Scheduled Query a BigQuery para que esta sentencia (CTAS) se ejecute automáticamente cada día a las 07:00 AM.**
- **» 3. Visualización (BI): conecta Looker Studio a tu tabla dim_transactions_flat y crea un Dashboard: “Monitor de rendiment de Vendes”.**

```
1 CREATE OR REPLACE FUNCTION `sprint3-analytics-borja-b.sprint3_golden.calculate_tax` (amount FLOAT64)
2 RETURNS FLOAT64
3 AS(
4   amount * 1.21
5 );
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Primero, creo una función con la query CREATE OR REPLACE FUNCTION que llamo calculate_tax, y le paso por parámetro el amount y FLOAT64. Retorno un dato del tipo FLOAT 64 que multiplique el amount * 1,21. De esta manera nos mostrará el precio con el 21% incrementado y, al haber sido creada en una función fuera de la tabla, en caso de necesitar modificar el IVA, tan solo será necesario modificar la función, y no buscar una a una cada tabla en la que ha sido creada.. Ejecuto y reviso que se haya creado:



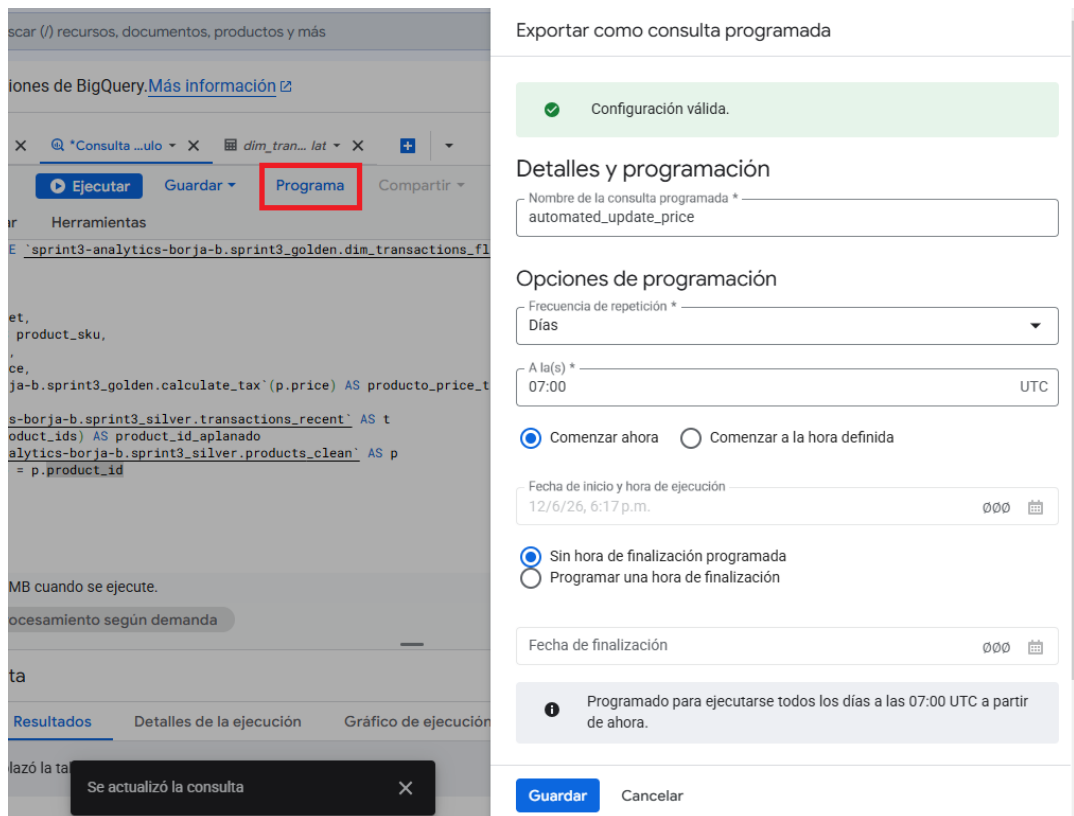
```
1 CREATE OR REPLACE TABLE `sprint3-analytics-borja-b.sprint3_golden.dim_transactions_flat` AS(  
2 SELECT  
3 t.transaction_id,  
4 t.timestamp,  
5 t.amount AS total_ticket,  
6 product_id_aplanado AS product_sku,  
7 p.name AS product_name,  
8 p.price AS product_price,  
9 `sprint3-analytics-borja-b.sprint3_golden.calculate_tax`(p.price) AS producto_price_tax_inc,  
10 p.category  
11 FROM `sprint3-analytics-borja-b.sprint3_silver.transactions_recent` AS t  
12 CROSS JOIN UNNEST(t.product_ids) AS product_id_aplanado  
13 INNER JOIN `sprint3-analytics-borja-b.sprint3_silver.products_clean` AS p  
14 ON product_id_aplanado = p.product_id  
15 WHERE t.declined = '0'  
16 );  
17
```

✓ Esta consulta procesará 7.37 MB cuando se ejecute.

Seguido, modifico la tabla del ejercicio 1 para añadir la función creada. Selecciono los campos, pero debajo del Price AS producto_price, añado la función para que la siguiente columna que muestre sea el precio incrementado, y lo llamo producto_price_tax_inc. Le añado la restricción para que solo muestre las transacciones que se han llevado a cabo. Ejecuto la query y compruebo que se ha creado la nueva columna en la tabla.

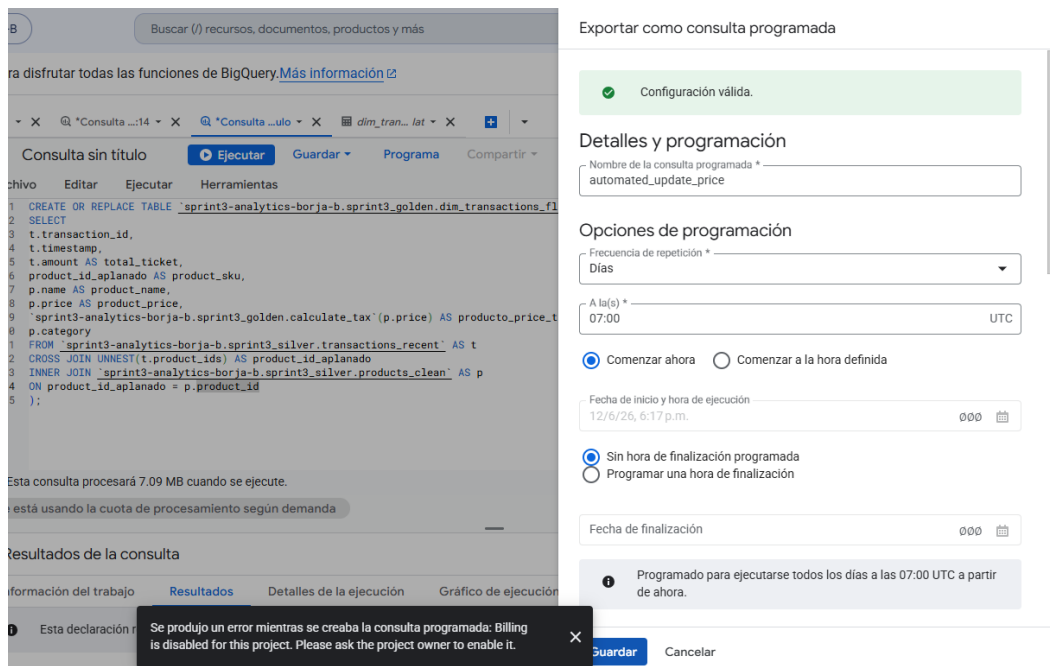
FILTRO Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción
☐	transaction_id	STRING	NULLABLE	-
☐	timestamp	TIMESTAMP	NULLABLE	-
☐	total_ticket	FLOAT	NULLABLE	-
☐	product_sku	INTEGER	NULLABLE	-
☐	product_name	STRING	NULLABLE	-
☐	product_price	FLOAT	NULLABLE	-
☐	producto_price_tax_inc	FLOAT	NULLABLE	-
☐	category	STRING	NULLABLE	-

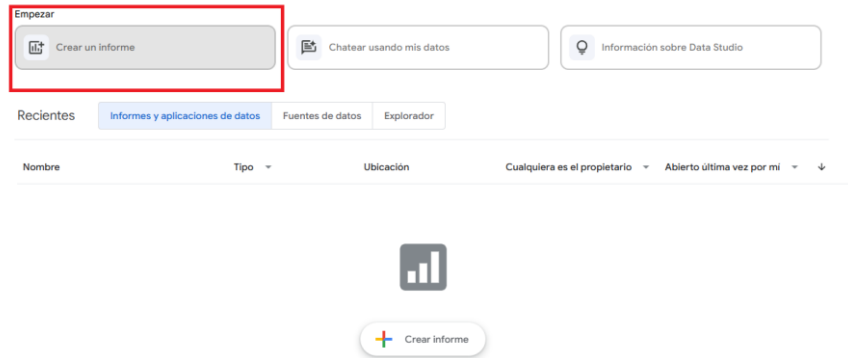


Por último, con el objetivo de actualizar cada día a las 7AM el precio con IVA, uso la opción del menú Programa, y le asigno un nombre “automated_update_price”. La frecuencia de la actualización es diaria a las 7 AM.

En la siguiente captura, se ve la ventana flotante indica que no tenemos los permisos suficientes de Billing, por tanto no es posible crear la programación.

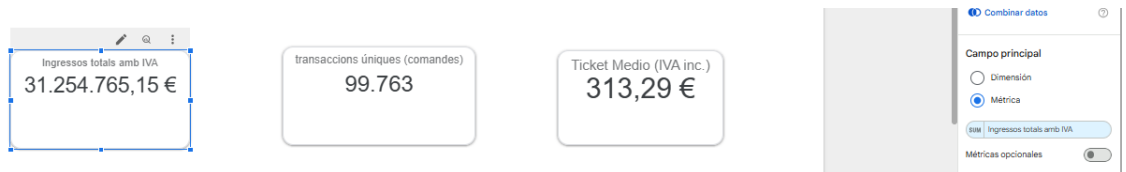


Para la visualización de los datos, entro en Looker Studio (actualmente Data Studio) y uso la opción crear un informe. Como fuente de datos usaré la tabla dim_transactions_flat.



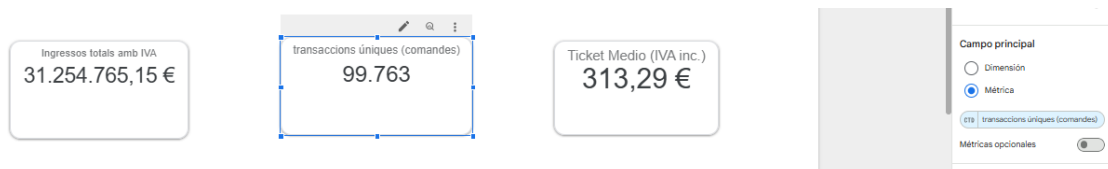
1- Los “Big Numbers”(KPI’s Generales)

En el menú superior elijo la opción añadir un gráfico → cuadro de resultados. Como métrica uso la suma de producto_price_tax_inc y renombro a Ingresos totales amb IVA.



Transacciones únicas (Pedidos)

Hago un count distinct de la columna transaction_id, y renombro a Transaccions úniques (comandes).



Transacciones únicas (pedidos)

En este caso, debo añadir un campo que llamo Ticket Medio (IVA inc), y hago la operación para obtener la media. En este caso, elijo la suma de producto_precio_tax_inc dividido entre el count distinct de transactions_id. Guardo y le doy a hecho y reviso que se haya creado.

dim_transactions_flat Ámbito: **insertada** Credenciales de acceso a datos: **Borja Belm** Actualización de datos: **12 horas** Acceso a visualizaciones comunitarias: **Activado** Funciones nativas: **Desactivadas** **Hecho**

← Todos los campos

Campos disponibles

- ABC category
- ABC product_name
- 123 product_price
- 123 product_sku
- 123 producto_price_tax_inc
- 123 timestamp
- 123 total_ticket
- ABC transaction_id
- 123 Record Count

Nombre del campo
 por ejemplo, Nuevo campo calculado
 Ticket Medio (IVA inc.)

ID de campo
 ID de campo
 calc_5kemattl4d

Fórmula
 1 SUM(producto_price_tax_inc)/COUNT(DISTINCT(transaction_id))

DAR FORMATO A FÓRMULA

Cancelar **Guardar**

Datos

 dim_transactions_flat

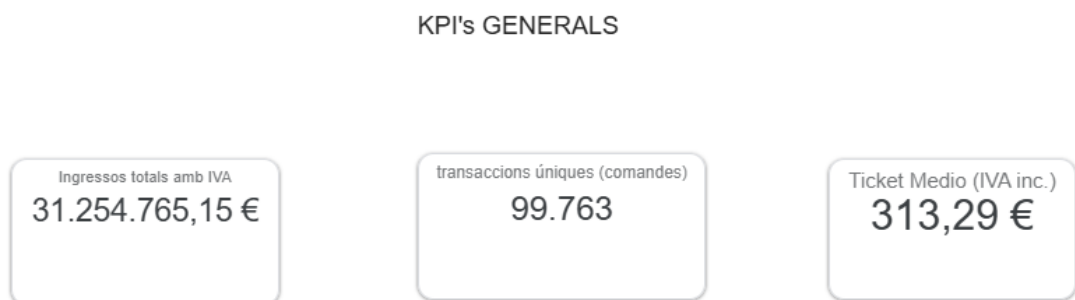
- ABC category
- ABC product_name
- 123 product_price
- 123 product_sku
- 123 producto_price_tax_inc
- 123 timestamp
- 123 total_ticket
- ABC transaction_id
- 123 Record Count
- 123 Ticket Medio (IVA inc.)**

Elijo como métrica el campo que acabo de crear y la uso:

Ticket Medio (IVA inc.)
313,41

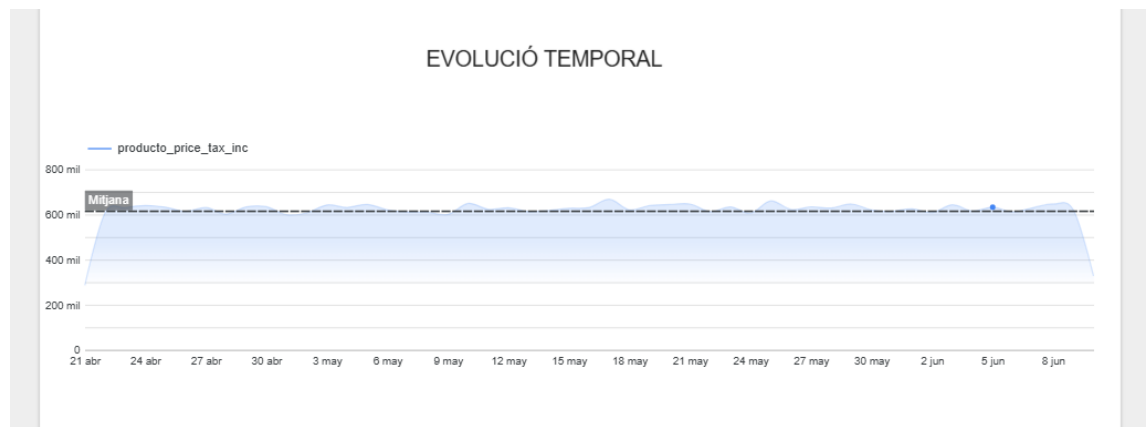
Campo principal
☐ Dimensión
☒ Métrica
 AUT Ticket Medio (IVA inc.)
 Métricas opcionales ☐

Resultado “Big Numbers” (KPI’s Generales):



2- Evolución temporal (Gráfico de series temporal)

Para esta gráfica, uso el campo timestamp como dimensión y desgloso por fecha. Como métrica uso la suma de product_price_tax_inc, y añado la línea de referencia media, de esta manera se puede ver más rápido en qué fechas las ventas han sido superiores a la media. El resultado es el siguiente:



3- Top productos Enriquecidos (tabla con Mapa de Calor)

Para esta tabla, elijo la tabla con mapa de calor, y elijo los siguiente campos:

Dimensión: product_name

Métrica 1: Cuento todo y renombro a “Unitats Venudes”

Métrica 2: hago la suma de product_price_tax_inc_ y renombro a “Ingressos totals”

Métrica 3: selecciono la opción de añadir un campo, y creo la métrica que suma el product_price_tax_inc y resta la suma de product_unit_price.

Una vez creada la métrica, este es el resultado:

dim_transactions_flat Ámbito: [Insertada](#) Credenciales de acceso a datos: [Borja Belm](#) Actualización de datos: [12 horas](#) Acceso a visualizaciones comunitarias [Activado](#) Funciones nativas [Desactivadas](#) [Hecho](#)

← Todos los campos

Campos disponibles

Q Search

- RIC category
- RIC product_name
- 123 product_price
- 123 product_sku
- 123 product_price_tax_inc
- 123 timestamp
- 123 total_ticket
- RIC transaction_id
- 123 Record Count
- 123 Ticket Medio (IVA inc.)

Nombre del campo
por ejemplo, Nuevo campo calculado
IVA recaptat

ID de campo
ID de campo
calc_nhvnanvi4d

Fórmula
SUM(product_price_tax_inc) - SUM(product_price)

[DAR FORMATO A FÓRMULA](#)

[Cancelar](#) [Guardar](#)

TOP PRODUCTES

	product_name	IVA recaptat	Unitats Venudes ▾	Ingressos Totals
1.	Winterfell	183.247,70 €	10.071	1.055.850,15 €
2.	the duel	221.209,7 €	10.059	1.274.589,23 €
3.	dooku solo	120.627,44 €	10.022	695.043,83 €
4.	Karstark Dorne	147.888,98 €	7.694	852.007 €
5.	skywalker ewok	287.102,64 €	7.607	1.539.019,97 €
6.	Karstark warden	218.533,92 €	7.563	1.247.647,83 €
7.	Direwolf Stannis	146.905,55 €	7.429	846.455,8 €
8.	duel Direwolf	150.842,43 €	5.167	887.987,33 €
9.	duel warden	154.706,95 €	5.166	891.406,69 €
10.	Lannister	98.094,8 €	5.155	565.212,9 €
11.	north	183.198,09 €	5.134	1.055.569,97 €
12.	Stark Karstark	119.228,4 €	5.125	688.982,69 €
13.	kingsblood Littlefinger the	80.895,54 €	5.124	486.112,38 €
14.	Tarly Stark	12.383,31 €	5.069	71.351,43 €
15.	Direwolf	182.823,28 €	5.064	1.053.410,32 €

1 - 70 / 70 < >

El mapa de calor, vemos que se ordena por unidades vendidas, sin embargo, lo que está diciendo es los productos que más ingresos totales aportan a la empresa, que como vemos, no tiene por qué ir en orden. En este caso el producto 7 Skywalker Ewok, a pesar de tener menos ventas, se obtiene más dinero a través del IVA del producto, y por tanto, los ingresos son superiores, en concreto, el producto 7 produce 6.774,59€ de ganancias más que el producto más vendido.

Visualización global final

KPI's GENERALS

Ingressos totals amb IVA
31.254.765,15 €

transaccions úniques (comand...
99.763

Ticket Medio (IVA inc.)
313,29 €

EVOLUCIÓ TEMPORAL



TOP PRODUCTES

	product_name	IVA reanptat	Unitats Venudes	Ingressos Totals
1.	Winterfell	162.587,38 €	10.041	1.052.051,08 €
2.	the duel	220.535,37 €	10.034	1.270.874,38 €
3.	cosoku woto	120.411,37 €	10.005	693.798,83 €
4.	Karlark Dome	147.383,27 €	7.888	849.208,37 €
5.	skywalker enrok	298.218,4 €	7.582	1.533.913,53 €
6.	Karlark warden	215.739,65 €	7.539	1.243.417,19 €
7.	Dinewoff Starna	148.816,24 €	7.421	845.941,19 €
8.	duel warden	154.487,23 €	5.158	890.025,45 €
9.	duel Dinewoff	150.138,97 €	5.151	885.088,48 €
10.	Lennaler	97.888,70 €	5.143	563.910,85 €
11.	north	182.953,54 €	5.127	1.054.180,88 €
12.	Stark Karlark	119.113,88 €	5.119	888.321,09 €
13.	kingeblood Lillalinger the	80.800,91 €	5.112	484.414,75 €
14.	Tarly Stark	12.350,12 €	5.084	71.180,2 €
15.	Dinewoff	162.355,93 €	5.051	1.050.740,54 €

1 - 70 / 70 < >

Link al documento:

<https://datastudio.google.com/reporting/26fa06f3-b3d3-4ee6-88c1-f6a7b654f2b1>